TP1 : Testeur de composants logiques / Bloc rangement des composants

Ce TP concerne une partie d'un bloc du système de gestion d'Emprunt.

Nous nous intéressons au bloc « Casier des composants logiques de la série 4000 ».

Ce module servant à aider au rangement des composants logiques dans le bon casier permet de vérifier leur bon fonctionnement en testant les fonctions logiques en enfichant le composant à tester dans un connecteur ZIF16.

Le bloc tiroir logique série 4000 comportera 16 casiers avec les références suivantes:

4000	4071
4001	4072
4011	4073
4023	4075
4030	4077
4048	4078
4068	4081
4069	4082
4070	4093

https://fr.wikipedia.org/wiki/Liste_des_circuits_int%C3%A9gr%C3%A9s_de_la_s%C3%A9rie_4000

1 Travail préparatoire : Bilan Entrees/Sorties de notre STM32

On ne va pas réaliser le bilan entrées/sorties complet du module, on va juste s'intéresser à:

- La gestion du bloc de 16 cases : On veut pour chaque casier pouvoir l'allumer en rouge ou en vert, et savoir s'il est ouvert ou fermé. On fera le bilan pour une case et on multiplie par le nombre de cases On utilisera des modules PCF8574 pour gérer les entrées sortie du bloc casier.
- La gestion du connecteur ZIF16 permettant de tester un composant via le montage proposé ci-dessous:

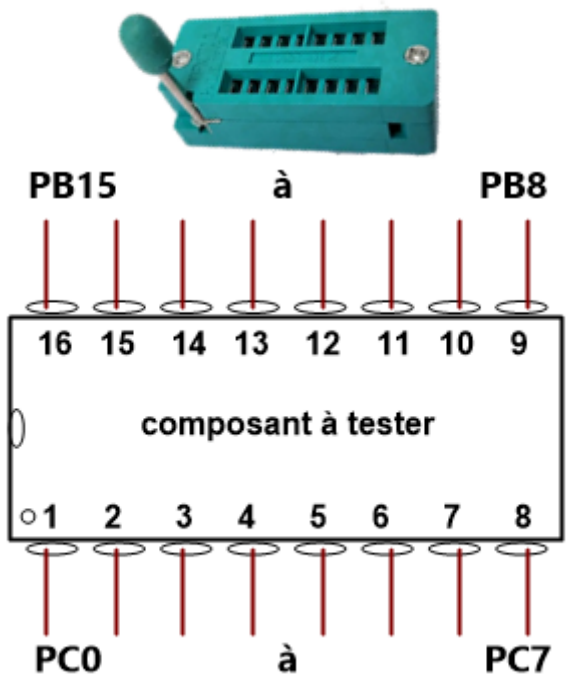
  A l'aide des Ports du STM32, on peut :

  - imposer des niveaux forts 0 et 1

    pour alimenter le composant.
  - imposer des niveaux faible H et L

    via les résistances de Pull_up ou de Pull_down internes du STM32 pour configurer les entrées

    du composant à tester.
  - lire l'état des pattes du composant, pour vérifier :
    - si une patte est une entrée (on relit toujours l'état du niveau logique faible proposé)
    - si la patte est une sortie (elle ne recopie pas les deux niveaux logiques faibles proposés)
    - analyser la logique du composant une fois les entrées et sorties identifiées et le composant alimenté



- Une liaison série bi directionnelle à 9600 bauds (celle connectée à l'usb) pour la mise au point:

  On identifiera les pattes concernées en regardant quelle est la liaison série envoyée au processeur qui gère l'USB dans les plans de la carte nucléo.

Avant de remplir le tableau ci-dessous, précisons que le pilotage des PCF8574 se fera en bit banging, et qu'ils seront tous pilotés ensemble, c'est leurs bits d'adresse qui permettra de les adresser individuellement.

module	nom	in/out/ ana/num	Pull up Pull down	Périphérique	PX_n	Commentaire
CASIER						
CASIER						
COM_ORDI						
COM_ORDI						
TESTEUR	comp_1				PC_	
	à				à	
	comp_8				PC_	
					PB_	
					à	
TESTEUR					PB_	

Le message série envoyé par le logiciel processing permettra au STM32 de configurer les 16 pattes du connecteur ZIF comme le souhaite l'ordinateur ou de changer l'état des leds des casiers. L'ordinateur enverra une séquence de 5 octets au STM32F103RB avec un protocole permettant de se resynchroniser dès qu'on reçoit l'entête du message:

- Seul l'octet d'entête contient un bit 7 à UN. Tous les octets suivants sont compris entre 0 et 127.
- l'entête contient tous les bits b7 des octets suivants : b0 = b7_octet1, b1 = b7_octet2, …..
- le bit B6 de l'entête permettra de distinguer deux trames :

  on récupérera une trame config testeur (si b6 = 0), et une trame config casier (si b6=1)

Trame Ordinateur → STM32 pour configurer les pattes du testeur		
octet	rôle	format
0	entête testeur (0x80) & bits b7 des octets suivants	0x80 | (b7_octet1<<0) |(b7_octet2<<1) \\ |(b7_octet3<<2) | (b7_octet4<<3) ;
1	direction pour les pattes PB15 à PB8 (dirB15 dans l'entete)	si la patte est en entrée le bit correspondant est à Un (1 = Input) sinon la patte est en sortie (juste les 2 alimentations) (0 = Output) en binaire : 0 dirB14 dirB13 dirB12 dirB11 dirB10 dirB9 dirB8
2	direction pour les pattes PC7 à PC0 (dirC7 dans l'entete)	si la patte est en entrée le bit correspondant est à Un (1 = Input) sinon la patte est en sortie (juste les 2 alimentations) (0 = Output) en binaire : 0 dirC6 dirC5 dirC4 dirC3 dirC2 dirC1 dirC0
3	etat logique voulu pour PB15 à PB8 (lvlB15 dans l'entete)	si la patte est une sortie c'est le niveau logique (juste les 2 alims) si la patte est une entrée cela indique L ou H(pull down ou pull up) en binaire : 0 lvlB14 lvlB13 lvlB12 lvlB11 lvlB10 lvlB9 lvlB8
4	etat logique voulu pour PC7 à PC0 (lvlC7 dans l'entete)	si la patte est une sortie c'est le niveau logique (juste les 2 alims) si la patte est une entrée cela indique L ou H(pull down ou pull up) en binaire : 0 lvlC6 lvlC5 lvlC4 lvlC3 lvlC2 lvlC1 lvlC0

Il faudra interpréter la trame et configurer les pattes en fonction. Le STM lira alors les pattes du testeur et fabriquera une trame de réponse. La trame renvoyée comportera 3 octets, entete 0x80 contenant l'état des deux bits B7 des octets suivants, suivi de deux octets codant l'état des 16 pattes (octet poids fort en premier).

Trame Ordinateur → STM32 pour configurer l'affichage rouge/vert des casiers		
octet	rôle	format
0	entête casiers (0xC0) & bits b7 des octets suivants	0xC0 | (b7_octet1<<0) |(b7_octet2<<1) \ |(b7_octet3<<2) | (b7_octet4<<3) ;
1	état des leds Vertes PB15 à PB8 (V_B15 dans l'entete)	pour les leds vertes allumées, le bit correspondant est à Un sinon les leds sont éteintes (pour chaque bit à Zéro). en binaire : 0 V_B V_B13 V_B12 V_B11 V_B10 V_B9 V_B8
2	état des leds Vertes PC7 à PC0 (V_C7 dans l'entete)	pour les leds vertes allumées, le bit correspondant est à Un sinon les leds sont éteintes (pour chaque bit à Zéro). en binaire : 0 V_C6 V_C5 V_C4 V_C3 V_C2 V_C1 V_C0
3	état des leds Rouges PB15 à PB8 (R_B15 dans l'entete)	pour les leds rouges allumées, le bit correspondant est à Un si les leds sont éteintes (pour chaque bit à Zéro). en binaire : 0 R_B R_B13 R_B12 R_B11 R_B10 R_B9 R_B8
4	état des leds Rouges PC7 à PC0 (R_C7 dans l'entete)	pour les leds rouges allumées, le bit correspondant est à Un sinon les leds sont éteintes (pour chaque bit à Zéro). en binaire : 0 R_C6 R_C5 R_C4 R_C3 R_C2 R_C1 R_C0

Il faudra interpréter la trame et allumer les leds en fonction. Attention, le PCF8574 ne sort pas de courant fort, par contre il peut absorber à niveau bas jusqu'à 25mA, les leds seront donc actives niveau bas. Le STM lira alors les PCF8574 qui interrogent les états des interrupteurs reed pour détecter les casiers ouverts et fabriquera une trame de réponse. La trame renvoyée comportera 3 octets, entete 0xC0 contenant l'état des deux bits B7 des octets suivants, suivi de deux octets codant l'état des 16 casiers ouverts ou fermés (octet poids fort en premier).

2 configuration des entrées-sorties :

2.1 Fonction de configuration

Pour faciliter le travail, vous allez créer une fonction qui permet pour une patte donnée d'un des trois ports A B ou C, de configurer la patte conformément à la demande passée en paramètre sous forme d'un #define.

Vous créerez une seconde version de cette fonction qui en plus configurera la pull up ou la pull down si on est en entrée ou appliquera le niveau logique si on est en sortie conformément à la demande d'un seul paramètre supplémentaire.

2.2 Initialisation des entrées sorties du projet :

2.2.1 Pattes de contrôle des PCF8574 des casiers :

Attention, bien qu'on n'utilise pas le périphérique I2C (on l'émule en BitBanging), il faut pourtant que les pattes choisies soient compatibles au niveau comportement hardware avec ce protocole.

2.2.2 Pattes reliées au ZIF16 :

On postule que l'on dispose de deux entiers 16bits, chaque bit associé à une patte: b0 = patte 1, b15 = patte 16

le premier entier informe si la patte est en niveau fort ou faible: bit à 0: patte en OUTPUT = niveau fort

le second entier informe si chaque patte est à niveau haut (1 ou H) ou bas (0 ou L) : chaque bit donne le niveau

On écrira une fonction qui met à jour l'état des 16 pattes en appelant les fonctions précédentes

A l'initialisation du processeur, quelle valeur mettre par sécurité dans nos deux entiers ?

2.2.3 Pattes liées à la liaison série :

On configurera les deux pattes permettant d'échanger avec l'ordinateur via le port série USB.

Il faut regarder sur la documentation de la carte Nucléo disponible sur moodle pour trouver ces pattes.

2.3 Initialisation des composants PCF8574 :

2.3.1 Ecrire des #define pour définir les pattes utilisées et les divers forcages :

On utilisera #define CLK_PCF 1<< __ et #define SDA_PCF 1<<__

ainsi on pourra facilement écrire des forcage à 0 et à H des pattes CLK et SDA du composant

2.3.2 Fonction émulant un start :

écrire la fonction qui émule un start, en agissant sur la patte CLK et la patte SDA. Attention aux timing

2.3.3 Fonction émulant un stop :

écrire la fonction qui émule un stop, en agissant sur la patte CLK et la patte SDA. Attention aux timing

2.3.4 Fonction écrivant un bit à 0 ou 1 passé en paramètre :

écrire la fonction qui pilote les pattes CLK et SDA pour transmettre un ZERO ou un UN.

2.3.5 Fonction renvoyant un bit à 0 ou 1 lu en i2C :

écrire la fonction qui pilote les pattes CLK et SDA pour pouvoir lirer un ZERO ou un UN et le renvoyer.

2.3.6 Fonction transmettant un octet passé en paramètre :

La fonction exploite les fonctions du dessus et devra renvoyer 0 ou 1 en fonction de l'acknowledge reçu

2.3.7 Fonction renvoyant un octet lu en I2C:

La fonction exploite les fonctions du dessus et devra renvoyer l'octet après avoir correctement acquitté le composant

2.3.8 Fonction permettant la mise à jour de 4 leds rouges/ vertes d'un PCF8574

Fonction permettant de configurer les 16 entrées capteurs :

Un composant PCF8574 va gérer les leds de 4 casiers (leds rouges sur bit pair, led verte sur bit impair suivant) l Le premier composant gère les casiers 0 à 3, le suivant les casiers 4 à 7 , le suivant, casiers 8 à 11 et le dernier gère l'allumage des casiers de 12 à 15. Vous devrez faire les séquences complètes pour écrire dans les composants concernés, chaque fonction renvoie 0 ou 1 en fonction de l'échec ou du succès de l'opération.

Attention le composant PCF8574 peut absorber de forts courants (25mA par sortie) mais n'en délivre que de très faibles (-100uA), les leds doivent être câblées pour s'allumer à niveau bas. De même pour pouvoir considérer une patte comme une entrée, il faut la configurer en sortie à niveau haut (pour profiter de la pull up)

2.3.9 Fonction permettant de lire l'état des 16 interrupteurs casiers :

Derrière chacun des 16 casiers, il y aura un interrupteur à lamelle établissant un contact entre la masse et l'entrée d'un composant PCF8574. Ces capteurs seront simulés par des dipswitch. Ecrire la fonction permettant de renvoyer l'état des 16 interrupteurs.

2.4 Configuration du périphérique de liaison série

On configurera la liaison série à 9600 bauds 1 bit de stop, pas de parité , pas de contrôle.

Pour tester si la configuration est bonne, on envoie en permanence par liaison série la valeur 0x55 et on vérifie à l'oscilloscope que la durée d'un bit est bien de 104us.

On peut tester alors avec un logiciel pour vérifier que l'on reçoit bien la lettre 'U'.

3 Passerelle Liaison série vers STM32.

3.1 Ecoute du port Série

Le but de cette partie est d'être capable de récupérer la consigne pour les 16 leds rouges et vertes,

de récupérer les consignes pour configurer les 16 pattes du processeur reliées au ZIF16

La technique employée est simple :

Il faut recherche quel bit change d'état dans quel registre à chaque réception d'un octet et lancer une routine qui va gérer le flot d'octet :

A chaque fois qu'on reçoit un octet, on regarde le registre de status pour voir s'il y a un soucis de communication ou pas.

En cas de soucis, on positionnera la machine à état en état d'attente de resynchronisation.

On récupère l'octet par lecture d'un registre, et dans le cas où il n y a pas de soucis de communication, on va vérifier si son bit de poids fort (b7) est à 1 ou 0.

Dans le premier cas, l'analyse du bit b6 permet de savoir si le message de 5 octets consécutifs va concerner les consignes pour les casiers ou pour les entrées sorties reliées au ZIF16, on va configurer la réception des octets suivant en mémorisant l'octet reçu dans la variable « entete » et en forçant une variable de machine à état MaeRcp à 1 ou 5. Il faudra penser à l aide des bits de poids faible de l'entête à remettre à jour les bits de poids fort des 4 octets suiveurs, bits forcés à zéro pour les distinguer de l'entête.

Dans le second cas, on va aiguiller le traitement à l'aide de la variable MaeRcp à l'aide d'une structure SWITCH CASE.

Le case 0 va servir d'état d'attente lors que l'on est perdu (comme au démarrage) en attente d'un entete.

Le case 1 va servir à réceptionner le premier octet de la consigne des casiers.

Les case 2 3 et 4 vont permettre de réceptionner les octets suivants.

Le case 5 va servir à réceptionner le premier octet de la consigne des entrées-sorties liées au ZIF16.

Les case 6 7 8 vont permettre de réceptionner les octets suivants.

Dans les case 4 et 8, on a la fin d'un message complet de 5 octets (entete et 4 octets), on forcera la machine à état sur le case 0 pour gérer des réceptions inattendues. On lèvera des flags pour déclencher les mises à jour dans le main, et on forcera la lecture des capteurs pour les casiers et la lecture des pattes pour le ZIF16

3.2 Envoi d'informations par port Série

On va maintenant répondre et renvoyer la trame de réponse.

Pour cela on met en œuvre une fifo d'envoi de 16 caractères :

On met en place un tableau Fifo[16] de 16 char, un index d'écriture IwFifo et de lecture IrFifo initialisés à 0, et enfin une variable FifoLibre initialisée à 16.

Après lecture des informations (soit les capteurs de casiers, soit les pattes du ZIF16) si la fifo comporte au moins 3 places libres, on écrit d'un seul coup le message à renvoyer.

Dans le main, si le périphérique de liaison série est disponible pour envoyer et que la fifo n'est pas vide, alors on va dépiler la fifo d'un élément.